Calculatorz

Arithmetic Expression Evaluator in C++
Software Requirements Specifications

Version 1.5

Revision History

Date	Version	Description	Author
2026-02-10	1.0	Drafted introduction	Isaiah + Group
2026-02-17	1.1	Drafted overall requirements	Isaiah + Group
2026-02-24	1.2	Drafted functionality	Isaiah + Group
2026-03-06	1.3	Completed draft	Eian + Group
2026-03-09	1.4	Formatting and polishing	Isaiah
2026-03-10	1.5	Final touches	Isaiah + Group

Table of Contents

Software Requirements Specifications

1. Introduction

1.1 Purpose

Our program will allow users to do basic calculator functions, such as all arithmetic operations. Utilizing valid inputs according to the standard order of operations, taking grouping symbols into account to alter precedence, and flagging invalid inputs to aid correction by the user.

1.2 Scope

The software will have a basic user interface for inputs and an error highlighter watching the whole program for thrown errors. Underneath will be a tokenizer module, a Prioritizer module, then an evaluator module; these will carry out the primary function of our program.

1.3 Definitions, Acronyms, and Abbreviations

See the Glossary in the *User's Manual*.

1.4 References

Project Description, Software Development Plan, *Software Architecture Document*, *Test Case*, and *User's Manual*.

1.5 Overview

This Software Requirements Specifications document captures the complete software requirements for the program. The following sections outline the project using use-case modeling.

2. Overall Description

2.1 Product perspective

2.1.1 User Interfaces

We plan on using Qt for the front-end design. This will enable us to create a product that is user-friendly and intuitive to use, and it can facilitate our detailed input error highlighting.

2.1.2 Software Interfaces

One of the many benefits of Qt is that the interface can be locally generated and run, so we would not have to rely on an internet connection. We can also use Qt to compile executables for several different operating systems.

2.1.3 Memory Constraints

The scope of our software should be small enough that we will not have to worry about memory limitations, however we will have all numerical values in C++'s `long double` datatype. This is relatively memory hungry for numerical datatypes, but will give us the most precision therefore reducing floating point errors. We also plan on keeping a persistent record of calculations, and this may take significant storage space.

2.2 Product functions

Generally, our software will perform all arithmetic. For our specific user base, we will implement random number generators themed as dice, and a persistent history of calculations performed.

2.3 User characteristics

Our calculator will be geared toward table-top role playing game (TTRPG) calculations.

2.4 Constraints

The user interface will allow us to limit the possible inputs, and the error highlighting can be used to deal with invalid inputs without crashing. Despite using the `long double` datatype, we are still constrained by the storage limits of it.

2.5 Assumptions and dependencies

We are depending on GitHub and Qt for development, but we plan on making the final product a self-contained executable that will function on a variety of operating systems.

3. Specific Requirements

3.1 Functionality

The core computation process in order is: Tokenization, Prioritization, and then Evaluation. Throughout the core process, the User Interface and Error Highlighting will update to reflect the state of the computation.

3.1.1 User Interface

The system should not allow input of any unused characters. Every valid character input will also be displayed on screen in an interactive keyboard, along with backspace and clear, and a submenu will have a list of constants. The interface should display the current expression and the result after submission. There will also be a menu option to show a brief history of evaluated expressions. The error highlighting should be integrated into the display of the current expression. Expressions that are longer than the input window should automatically scroll the string to the left, keeping the input right-justified.

3.1.2 Error Highlighting

This function will be integrated into the whole computation process using throwing and catching capabilities to identify errors and their causes. The highlighting will be integrated into the interface to show which character caused the error. Then remove the unexpected tokens, add expected tokens, and continue to evaluate until a fatal error is found or a result can be reached.

3.1.3 Tokenization

This module will capture valid input strings and differentiate between unary and binary operators with the same symbol. It will throw errors with a description that the Error Highlighting module can use.

3.1.4 Prioritization

When evaluating prioritization, an error should be thrown with mismatched parentheses along with a description that the Error Highlighting module can use to assume a valid prioritization.

3.1.5 Evaluation

The program needs to accurately compute arithmetic operations.

3.2 Use-Case Specifications

Our primary use case will be getting the results of computations used during tabletop role-playing games. This includes dice rolling of standard tabletop roleplaying game dice (having 4, 6, 8, 10, 12, 20, and 100 sides). Alongside computing any kind of arithmetic the user may input.

3.3 Supplementary Requirements

3.3.1 Locally Executed

Works offline, without an internet connection.

3.3.2 Efficient

Computes operations and performs randomized dice rolls while taking a reasonable amount of time and memory.

3.3.3 Self-Contained

Will not create supplementary files in the same directory as the executable and will uninstall seamlessly.

3.3.4 Multi-Platform

The program should be functional on different operating systems such as MacOS, ChromeOS, Ubuntu, and Windows.

4. Classification of Functional Requirements

Functionality	Type
User Interface	Desirable
Error Highlighting	Essential
Tokenization	Essential
Prioritization	Essential
Evaluation	Essential
Locally Executed	Essential
Efficient	Desirable
Self-Contained	Desirable
Multi-Platform	Optional

5. Appendices

See the *Software Architecture* document for the specific implementation of the requirements.